Architettura degli Agenti LLM: dalla struttura base al workflow operativo

---

1. Premessa

Il campo dell'AI evolve rapidamente. "Intelligenza artificiale", "modello generativo", "LLM", "agente", "RAG", "MCP", "tool", "framework" — spesso usati come sinonimi, come gergo da conferenza, come marketing. Il risultato è disorientamento anche in contesti dove il vocabolario tecnico sarebbe sufficiente a fare chiarezza.

Questo documento costruisce una mappa mentale dall'architettura di base fino al workflow operativo, nell'ordine in cui ogni livello diventa necessario per capire il successivo. L'obiettivo non è una survey accademica né una guida all'installazione. È avere le fondamenta per valutare qualsiasi nuovo strumento o framework che emerga nel campo.

Una premessa onesta: il campo si muove rapidamente. Quello che oggi è framework di riferimento potrebbe essere obsoleto domani. La struttura di base, invece, è stabile — ed è su quella che vale la pena investire tempo.

La tesi centrale di questo documento: la struttura invariante di un agente — LLM + loop + memoria + strumenti — è più importante di qualsiasi framework specifico. Capire quella struttura permette di valutare ogni nuovo tool che emerga nel campo, invece di rincorrere la novità.

---

2. Il Transformer: cosa c'è sotto il cofano

2.1 Il problema che risolveva

Prima del 2017, il processing del linguaggio avveniva attraverso reti ricorrenti (RNN). Il principio era intuitivo: leggere parola per parola, mantenendo uno stato interno che accumulasse contesto. Il limite era strutturale: lo stato si comprimeva, il gradiente svaniva nelle sequenze lunghe, le dipendenze a distanza tra parole si perdevano. Una frase come *"Il server che gestisce il cluster su cui gira l'applicazione deployata ieri è caduto"* — soggetto e verbo separati da dodici parole — era già un problema serio.

2.2 L'attenzione come soluzione

Il paper *Attention Is All You Need* (Vaswani et al., 2017) propose un approccio diverso: invece di processare le parole in sequenza, ogni token osserva tutti gli altri contemporaneamente, e il modello impara quanto peso dare a ciascuna relazione. Questo meccanismo si chiama **self-attention**.

In termini operativi: ogni token produce tre vettori — Query, Key, Value. La compatibilità tra la Query di un token e le Key di tutti gli altri determina l'attenzione distribuita. Il risultato è una rappresentazione contestuale: "banco" in "banco di pesce" e in "banco dati" genera vettori distinti perché il contesto circostante orienta il significato diversamente.

Questo meccanismo è parallelizzabile. Parallelizzabile significa GPU. GPU significa scala. Scala significa dataset enormi. Training su dataset enormi porta alla nascita degli LLM.

2.3 Cosa impara un LLM

Addestrare un Transformer su centinaia di miliardi di token con un obiettivo apparentemente banale — predire il token successivo — produce un risultato che non era interamente previsto: il modello non acquisisce solo sintassi. Acquisisce semantica, ragionamento analogico, strutture logiche, conoscenze enciclopediche, convenzioni di dominio.

Il meccanismo è diretto: predire correttamente il token successivo *richiede* di comprendere il contesto profondo del testo. Per sapere che dopo "La capitale della Francia è" viene "Parigi", il modello deve aver interiorizzato una mappa geografica implicita. Per completare un argomento logico coerente, deve aver astratto la struttura dell'argomentazione.

La conseguenza fondamentale è questa: **la semantica è implicita nel modello, non esplicitata in alcuna struttura formale**. Non c'è un knowledge graph, non c'è un'ontologia, non ci sono regole logiche dichiarate. C'è un enorme spazio vettoriale dove concetti semanticamente simili occupano regioni spazialmente vicine, e dove le relazioni tra concetti sono codificate come direzioni geometriche.

2.4 I limiti strutturali

Un LLM da solo è un oracolo senza memoria e senza capacità di azione. Ha tre limiti che non può superare autonomamente:

- **Nessuna memoria persistente.** Ogni sessione inizia da zero. Il contesto delle sessioni precedenti non esiste.
- **Nessuna capacità di agire nel mondo.** Non legge file, non chiama API, non modifica sistemi. Produce testo.
- **Finestra di contesto finita.** La quantità di testo osservabile in una singola sessione è limitata — nell'ordine di 100K–200K token per i modelli più diffusi (Claude, GPT-4), fino a 1M+ token per modelli specifici come Gemini 2.0. Una categoria emergente — i **reasoning**

models (o1, DeepSeek R1, Gemini 2.5 Thinking) — esegue ragionamento interno prima di rispondere, riducendo il numero di iterazioni necessarie nel loop ReAct ma aumentando latenza e costo per chiamata; questo rappresenta un nuovo tradeoff per la scelta del cervello agente.

Questi non sono bug. Sono i confini architetturali del componente. Capire questi confini è il punto di partenza per capire cosa fanno gli agenti — e in particolare per capire perché il problema della memoria, trattato nel capitolo 4, non ha una soluzione banale.

---

3. L'Agente: quando l'LLM inizia ad agire

3.1 La definizione più precisa

Un agente è un **while loop con un LLM come cervello**.

Un agente riceve un obiettivo, chiede all'LLM cosa fare, esegue l'azione scelta, osserva il risultato, reinserisce l'osservazione nel contesto, e ricomincia. Il ciclo continua finché l'obiettivo non è raggiunto o il loop viene interrotto esplicitamente.

Il pattern formale si chiama **ReAct** (Reason + Act):

```
┌──────────────────────────────────────────────────────────┐
│                    CICLO AGENTE                          │
├──────────────────────────────────────────────────────────┤
│                                                          │
│  OBIETTIVO                                               │
│      ↓                                                   │
│  [LLM RAGIONA] — Cosa devo fare?                         │
│      ↓                                                   │
│  [SCEGLI AZIONE] — Quale tool invocare?                  │
│      ↓                                                   │
│  [ESEGUI] — L'ambiente esegue la funzione                │
│      ↓                                                   │
│  [OSSERVA IL RISULTATO] — Condividi l'output con LLM     │
│      ↓                                                   │
│  Obiettivo raggiunto? ──NO──→ [LLM RAGIONA di nuovo]     │
│      │                                                   │
│     SÌ                                                   │
│      ↓                                                   │
│  FINE LOOP                                               │
│                                                          │
└──────────────────────────────────────────────────────────┘
```

La differenza rispetto a una singola chiamata LLM è il ciclo. L'agente non risponde una volta. Persiste, agisce, corregge il tiro sulla base di quello che osserva.

3.2 Un agente in azione: esempio concreto

Obiettivo: **"Leggi il file config.json, cambia il valore di `debug` da `false` a `true`, e salva il file".**

L'LLM non ha accesso diretto al file system. Ha accesso a funzioni (tool). L'agente cicla attraverso il ReAct pattern:

Iterazione 1 — Il problema

- LLM legge l'obiettivo
- LLM ragiona: "Non conosco il contenuto di config.json. Devo leggerlo per sapere cosa modificare."
- LLM invoca: `read_file("config.json")`
- Ambiente esegue il comando e ritorna il contenuto

Iterazione 2 — L'osservazione

- LLM riceve il contenuto:

```
{  "debug": false,  "port": 3000}
```

- LLM ragiona: "Vedo che `debug` è `false`. Devo cambiarlo a `true` e salvare."
- LLM invoca: `write_file("config.json", new_content)`
- Ambiente esegue il comando e ritorna successo

Iterazione 3 — Verifica

- LLM ragiona: "Ho scritto il file. Verifico che sia stato salvato correttamente."
- LLM invoca: `read_file("config.json")`
- Ambiente ritorna il nuovo contenuto con `debug: true`
- Obiettivo raggiunto. Loop termina.

La sequenza completa:

```
┌──────────────────┐     ┌────────────────┐     ┌──────────────┐
│     Obiettivo    │────→│  LLM ragiona   │────→│   Invoca     │
│                  │     │  cosa fare?    │     │    tool      │
└──────────────────┘     └────────────────┘     └──────────────┘
                                                        │
        ↑                                               │
        │                                               ↓
        │                ┌────────────────┐     ┌──────────────┐
```

```
        │               │ Osserva il   │←───│  Ambiente   │
        └───────────────│  risultato   │    │  esegue     │
                        └──────────────┘    └─────────────┘
```

Ogni iterazione è una decisione autonoma dell'LLM basata su ciò che osserva. Non c'è logica hardcoded: l'LLM decide il prossimo step per raggiungere l'obiettivo.

3.3 Strumenti: function calling e MCP

Un agente non può operare senza **strumenti** — le funzioni che può invocare. Nel termine tecnico: *function calling* o *tool use*. Il modello, addestrato a produrre output strutturati, genera chiamate di funzione. Il programma che lo avvolge intercetta queste chiamate, le esegue, restituisce il risultato al modello nel ciclo successivo.

Fino al 2024, ogni framework definiva il proprio protocollo per esporre gli strumenti. Nel novembre 2024, Anthropic ha lanciato il **Model Context Protocol (MCP)**: uno standard aperto per la comunicazione tra LLM e strumenti esterni. Già nella prima metà del 2025 OpenAI e Google DeepMind ne hanno annunciato il supporto. L'adozione è reale, anche se il campo rimane parzialmente frammentato: i protocolli proprietari coesistono con MCP, non sono stati sostituiti. L'analogia utile è quella di USB-C: prima della standardizzazione, ogni produttore aveva il proprio connettore. MCP punta a replicare quella logica — uno strumento scritto una volta, compatibile con qualsiasi LLM che lo supporti.

---

4. La Memoria: il problema aperto

4.1 Tre dimensioni del problema

Un LLM non ha stato tra sessioni. Come illustrato nella sezione 2.4, la finestra di contesto è finita e ogni nuova conversazione ricomincia da zero. Per un chatbot generalista, questo è accettabile. Per un sistema che opera su una base di conoscenza in crescita, non lo è.

Il problema della memoria negli agenti si articola in tre dimensioni:

- **Cosa memorizzare**: non tutto è rilevante a lungo termine. La selezione è non banale.
- **Come organizzarlo**: la struttura della memoria influenza la qualità del ragionamento futuro. Una lista piatta di fatti è meno utile di una struttura con relazioni esplicite.
- **Come recuperarlo**: la finestra di contesto è finita. Non è possibile iniettare tutta la memoria in ogni sessione. Bisogna scegliere cosa è pertinente al task corrente.

4.2 Le tre tipologie di memoria

Indipendentemente da come implementi un agente — database, vector store, file system, memoria in-memory — il problema della memoria si scompone sempre in tre categorie:

Tipo	Durata	Mutabilità	Funzione	Esempi di realizzazione
Contestuale	Solo nella sessione corrente	Sì (si accumula)	Buffer temporaneo dove conversazione e ragionamenti si accumulano durante il ciclo	Finestra di contesto dell'LLM, conversation history, output di tool passati al modello
Statica	Permanente	No	Regole, istruzioni, metodi che guidano il comportamento — scritti una volta, non cambiano	System prompt, regole aziendali, istruzioni di dominio, librerie di procedure, configurazioni
Episodica	Permanente	Sì	Storia operativa, decisioni prese, eventi osservati — accumulata nel tempo e richiamabile	Vector database, log strutturato, memory buffer, transcript di sessioni passate, knowledge graph

Come funzionano insieme:

- **Contestuale** è il "qui e ora" — tutto quello che succede durante questa sessione. Non persiste oltre la sessione stessa.
- **Statica** è il "manuale operativo" — come dovrebbe comportarsi l'agente, cosa sa fare, quali vincoli rispetta. Non cambia (o cambia raramente, solo se deliberatamente aggiornata).
- **Episodica** è il "diario" — che cosa è già stato fatto, quali decisioni sono state prese, quali fatti emergono dal lavoro. Accumula nel tempo e viene ripescata nelle sessioni successive sulla base della rilevanza.

L'arte di progettare un agente sta nel decidere: cosa finisce in quale categoria, e come viene recuperato e iniettato nel contesto quando necessario.

4.3 Le skill: prompting strutturato e automatizzabile

Alcune operazioni si ripetono ogni giorno chattando con AI: dare contesto del progetto, prompt per analizzare un documento, strutturare l'output desiderato, guidare l'agente nel ragionamento. Una **skill file** fissa queste istruzioni in Markdown e le rende ripetibili — è la realizzazione concreta della **memoria statica** descritta sopra.

Le skill non sono codice. Sono **procedure di prompting condensate**: descrivono il *come* affrontare un problema ricorrente. Esempi concreti:

- Una skill `ingest-paper.md` dice all'agente: "quando ricevi un paper, leggi il file analisi, identifica i concetti chiave, cercali nella wiki, aggiorna le pagine correlate, registra nel log."
- Una skill `warm-up-context.md` (caso d'uso comune) dice: "prima di iniziare lavoro vero, carica il contesto — leggi `CLAUDE.md` per il ruolo, leggi `wiki/scaffolding.md` per lo stato del progetto, leggi `wiki/log.md` per le ultime operazioni, riassumi all'utente dove siamo."

Poiché le skill sono **solo istruzioni strutturate**, chiunque può leggerle, adattarle e contribuire versioni migliori. Sono nati diversi hub in cui vengono condivise (come Vercel Agent Resources e Anthropic Prompt Library). Il workflow pratico in cui le skill vengono usate concretamente è descritto nel capitolo 6.

4.4 RAG vs. iniezione diretta di Markdown

RAG (Retrieval-Augmented Generation) è l'approccio prevalente nell'industria: i documenti vengono indicizzati in un vector store; al momento della query, i chunk semanticamente più vicini vengono recuperati e inseriti nel contesto. Funziona bene con migliaia o milioni di documenti che non possono essere iniettati interamente.

Il limite del RAG non è tecnico, è cognitivo: si recuperano frammenti, non sintesi. L'LLM riceve pezzi sconnessi di conoscenza, non una mappa organizzata. La qualità del ragionamento dipende dalla qualità del retrieval, che dipende dalla qualità dell'embedding — non perfetta, specialmente su concetti tecnici specialistici.

L'alternativa Markdown parte da una premessa diversa: invece di recuperare da documenti grezzi al momento della query, si costruisce e mantiene una **wiki sintetica** — file Markdown strutturati e intercollegati, che l'agente aggiorna man mano che arrivano nuovi input. Ogni nuova sessione riceve questa wiki già elaborata, non i documenti originali.

Nel linguaggio della sezione 4.2, il pattern wiki realizza:

- **Memoria statica** → file di skill e linee guida (`AGENT.md` , procedure in Markdown)
- **Memoria episodica** → wiki sintetizzata + log (tutto accumulato su file e ripescato nelle sessioni successive)
- **Memoria contestuale** → conversazione in corso

Il risultato è **memoria che compila, non che recupera**. La conoscenza si accumula e si raffina invece di restare frammentata in chunk non elaborati.

Recentemente (aprile 2026), Andrej Karpathy ha formalizzato questo pattern con il concetto di **LLM Wiki**: due livelli separati — una cartella `raw` con i documenti originali immutabili, e una cartella `wiki` interamente gestita dall'LLM, con pagine per concetti, entità, sintesi, cross-

reference. Quando arriva un nuovo documento, l'agente non lo indicizza: lo legge, lo integra nella wiki, aggiorna le pagine esistenti, nota le contraddizioni con il materiale precedente.

Il limite pratico: scala bene fino a centinaia di documenti. Per dataset di scala aziendale, RAG rimane necessario. Per workflow personali o di team su domini circoscritti, l'approccio wiki produce ragionamento di qualità superiore con infrastruttura zero.

4.5 Il palazzo mentale digitale: un approccio ibrido

> *Ricerca emergente — non ancora in produzione matura*

Parallelo al pattern wiki, emerge un approccio ispirato al **palazzo mentale** (metodo dei Loci) degli antichi greci. L'idea: invece di una flat list di documenti o chunk semanticamente recuperati, la memoria viene organizzata come una **gerarchia spaziale virtuale** — Ali (progetti) → Sale (categorie) → Stanze (idee specifiche). Progetti open-source come MemPalace sperimentano questa direzione.

Elementi tecnici distintivi:

- **Compressione AAAK**: un formato deterministico (circa 30x di riduzione) che permette di rappresentare settimane di conversazioni in pochi hundred token, leggibile nativamente da qualsiasi LLM.
- **Local-first**: a differenza di RAG che chiama API esterne per il retrieval, usa regole locali e SQLite, veloce e offline.
- **Grafo temporale + Vector Store**: combina ChromaDB (per vicinanza semantica) e SQLite (per relazioni temporali).

Il limite è la complessità: più sofisticato di un wiki puro, richiede tooling dedicato e una disciplina rigorosa nel mantenimento della struttura spaziale. Per natura, è più simile a **RAG che a wiki** — la memoria "vive" in uno strato di elaborazione esterno (lo spazio virtuale), anche se local-first.

4.6 Posizionamento dei tre approcci

I tre approcci si posizionano lungo tre dimensioni: **scala del corpus**, **struttura della conoscenza**, e **infrastruttura richiesta**.

Dimensione	Markdown / Wiki	RAG	Palazzo Mentale
Numero documenti	Decine–centinaia	Migliaia–milioni	Centinaia (frontiera)
Dove vive la memoria	File system locale, Git-tracked	Vector store esterno	SQLite + ChromaDB locali

Dimensione	Markdown / Wiki	RAG	Palazzo Mentale
Struttura della conoscenza	Gerarchica per file, collegamento esplicito via link `[[page]]`	Flat semanticamente indicizzata, nessuna gerarchia esplicita	Gerarchica per spazio (Ali → Sale → Stanze)
Recupero info	Compilativo: sintetizza al momento dell'ingestione	Semantico: query genera embedding, cerca chunk simili	Ibrido: navigazione spaziale + similarità vettoriale
Qualità del ragionamento	Alta: sintesi strutturate, non frammenti	Media-alta: dipende dalla qualità del retrieval	Alta potenziale: richiede disciplina
Complessità d'implementazione	Minimale: file Markdown, Git, bash	Media-alta: vector DB, embedding, chunking	Media-alta: tooling dedicato
Infrastruttura da gestire	Zero (file system)	Vector DB + embedding API	SQLite + ChromaDB locali

La progressione:

Markdown/Wiki è la soluzione per flussi locali, controllati, di scala personale o di team. Il valore viene dal compilare conoscenza mentre viene ingestita, non dal retrieval. Il limite è al confine di 200–300 documenti densi.

RAG entra quando la scala cresce e il corpus è parzialmente incontrollato. Lo scambio è: si perde qualità di ragionamento (frammenti non contestuali), si guadagna scalabilità.

Palazzo Mentale è frontiera di ibridazione: prende la struttura gerarchica del wiki e la semantica del RAG. È ancora esplorazione, non produzione matura.

---

5. I Framework: il mercato impacchetta il blob

5.1 La situazione reale

Nel 2025–2026 il campo degli agenti è, per usare un termine tecnico, un **blob non ancora standardizzato**. Esistono pattern che funzionano, best practice emergenti, componenti ricorrenti — ma non esiste ancora un equivalente di quello che TCP/IP fu per il networking o di quello che il browser fu per il web. Ogni framework fa scelte diverse su orchestrazione, gestione della memoria, parallelismo, osservabilità.

In questo contesto, ogni framework risolve lo stesso problema in modo diverso: prendere i componenti base (LLM + loop + strumenti + memoria) e renderli utilizzabili senza dover riscrivere la stessa infrastruttura da zero.

5.2 I protagonisti del mercato

Framework	Target	Filosofia	Punto di forza	Limite
LangChain	Developer	Swiss Army knife	Ecosistema vasto, centinaia di connettori	Astrazioni ridondanti, over-engineering facile
LangGraph	Developer	Grafi di esecuzione	Controllo fine sul flusso, agenti stateful	Curva di apprendimento ripida
CrewAI	Developer	Team di agenti con ruoli	Setup intuitivo, multi-agent out of the box	Ecosistema più giovane
Claude Code	Sviluppatore singolo	CLI + file system nativo	Trasparenza sulle azioni, controllo diretto	Legato all'ecosistema Anthropic
smolagents (HuggingFace)	Ricerca / Minimalist	API-first minimalista	Capire la base prima di adottare un framework	Nessun guardrail di produzione
Pi (pi.dev)	Developer Minimalisti	Terminal harness aggressivamente minimale (4 tool: Read, Write, Edit, Bash)	Auto-estensibile in TypeScript, velocità estrema, zero overhead, massimo controllo	No MCP nativo (per scelta filosofica: mantenere core puro); solo CLI
OpenClaw	Utenti finali / Power User	Personal AI Assistant via messaggistica nativa (WhatsApp, Telegram, Discord) per compiti quotidiani	Interfaccia naturale, memoria 24/7, auto-miglioramento via skill Markdown, local-first data sovereignty	Costruito su SDK Pi; complessità di setup iniziale; richiede consapevolezza sicurezza
NVIDIA NemoClaw™	Enterprise	OpenClaw blindato (hardened); GPU	Robustezza industriale, modelli	Richiede hardware NVIDIA; licenze

Framework	Target	Filosofia	Punto di forza	Limite
		optimization; sandbox di sicurezza (OpenShell)	Nemotron specializzati, guardrail integrati, osservabilità real-time	Enterprise; overhead operativo significativo

5.3 La struttura invariante

Togliendo il packaging, tutti i framework riducono alla stessa struttura di base:

```
┌──────────────────────────────────────────────────────────┐
│                    AGENTE                                 │
│                                                           │
│        LLM (GPT-4, Claude, Llama...)                     │
│                 ↑          ↓                              │
│            [contesto]  [azione scelta]                   │
│                 ↑          ↓                              │
│          ┌────────────────┬──────────────┐               │
│          │  Memoria MD    │  Strumenti   │               │
│          │  (skill +      │  (file I/O,  │               │
│          │  episodica)    │  API, MCP)   │               │
│          └────────────────┴──────────────┘               │
└──────────────────────────────────────────────────────────┘
```

Tre componenti, ognuno sostituibile indipendentemente:

1. **LLM** — il cervello. Cloud (GPT-4o, Claude) o locale (Llama, Mistral). La logica dell'agente non cambia al cambiare del modello.
2. **File Markdown** — la memoria. Skill file per il *come fare*, file episodici per il *cosa è successo*. Leggibili dall'uomo, scrivibili dall'agente.
3. **Interfaccia strumenti** — le braccia. File system, API esterne, terminale. Standardizzabile via MCP.

5.4 Perché i framework esistono

I framework non nascono da un'esigenza tecnica irrisolta. Nascono per due ragioni convergenti.

Dal lato commerciale: un blob di tecnologia potente ma non standardizzato non è vendibile direttamente. Serve packaging — una superficie d'uso stabile, documentazione, supporto, un nome riconoscibile.

Dal lato dell'usabilità: ogni software, per raggiungere un pubblico ampio, ha bisogno di un'interfaccia. Un database ha un query language. Un sistema operativo ha una shell o una GUI. Un agente AI, data la generalità delle sue possibili applicazioni, non ha un'interfaccia di dominio ovvia. Il risultato, quasi invariabilmente, è una **CLI o una chat** dove l'agente gestisce tutto il lavoro operativo sotto la superficie. L'interazione si riduce a: descrivi cosa vuoi → l'agente esegue.

La struttura base (agente + LLM + file Markdown) è già completa per uso diretto. I framework aggiungono guardrail, retry logic, osservabilità — valore reale in produzione, non necessario nella sperimentazione. La comprensione del campo passa prima dalla struttura, poi eventualmente dal framework — non viceversa.

---

6. Il Workflow Pratico: Wiki Tesi

6.1 La struttura delle cartelle

Il pattern LLM Wiki applicato a un workflow operativo reale prende questa forma:

```
raw/                                    ← immutabile, solo lettura
├── papers/
│   └── attention-paper/
│       ├── paper.pdf
│       └── valore.md                  ← analisi sintetizzata
├── calls/
│   └── call-2026-04-15.md
└── project/
    └── approfondimenti/
        └── tema-agenti.md

wiki/                                   ← interamente gestita dall'agente
├── index.md                            ← catalogo: punto di ingresso
├── log.md                              ← append-only: traccia ogni azione
├── scaffolding.md                      ← documento centrale: stato progetto
├── glossary.md                         ← terminologia canonica
├── sources/                            ← una pagina per ogni documento
└── concepts/                           ← una pagina per ogni concetto

CLAUDE.md                        ← manuale operativo + contesto
skills/
├── ingest-paper.md              ← skill: procedura ingestione paper
└── warm-up-context.md           ← skill: bootstrap sessione
```

La separazione `raw/` vs `wiki/` è la realizzazione concreta del pattern Karpathy: i documenti originali non vengono mai modificati, la wiki è la loro elaborazione strutturata e cumulativa.

6.2 L'agente in bash: dallo schema astratto alla realtà

Nella sezione 3.2 l'esempio usava notazione astratta (`read_file()` , `list_files()`). Concretamente, un agente con accesso al terminale usa bash ordinario — gli stessi comandi che usa qualsiasi developer:

```
# ── BOOTSTRAP SESSIONE ──────────────────────────────────────
cat skills/AGENT.md                    # carica ruolo & contesto
cat wiki/index.md                      # orienta nella wiki
tail -n 50 wiki/log.md                 # ultime attività

# ── RICOGNIZIONE ────────────────────────────────────────────
ls raw/papers/attention-paper/
cat raw/papers/attention-paper/valore.md

# ── VERIFICA CORRELAZIONI ───────────────────────────────────
grep -rl "transformer" wiki/concepts/ wiki/sources/
cat wiki/concepts/transformer.md

# ── SCRITTURA ───────────────────────────────────────────────
cat > wiki/sources/attention-paper.md << 'EOF'
...contenuto generato dall'LLM...
EOF

# ── AGGIORNAMENTI ───────────────────────────────────────────
echo "..." >> wiki/concepts/transformer.md
echo "..." >> wiki/log.md
```

Non c'è astrazione: l'agente vede file, li legge con `cat` , cerca pattern con `grep` , ne crea con redirezione. Le `tool_function()` del modello teorico corrispondono esattamente a comandi bash. Il loop ReAct è il ciclo: osservo l'output del comando → decido il prossimo → eseguo.

6.3 Il file che porta l'agente al contesto

Il file `CLAUDE.md` (o `AGENT.md`) è la realizzazione concreta della **memoria statica** descritta nella sezione 4.2. Non è un prompt scritto ogni volta a mano — è un documento Markdown che l'agente legge all'inizio di ogni sessione e che risponde a: chi sono, cosa gestisco, dove si trovano i file, quali workflow seguire, quali regole non violare mai.

Una skill file come `ingest-paper.md` è più granulare: descrive la procedura esatta per un'operazione specifica — quali file leggere per prima cosa, come determinare il caso (paper

già analizzato o da analizzare da zero), come aggiornare ogni pagina coinvolta, cosa scrivere nel log. Non c'è logica hardcoded nel programma: la logica è nel Markdown, leggibile e modificabile da chiunque senza toccare codice.

Una sessione tipica:

1. **Sessione avvia.** L'LLM riceve come primo input: "Leggi `CLAUDE.md` per orientarti."
2. **Contesto caricato.** `CLAUDE.md` contiene: il tuo ruolo, il progetto su cui lavori, lo stato attuale (da `wiki/scaffolding.md`), storia recente di cosa è stato fatto (`wiki/log.md`).
3. **Geometria del progetto.** L'LLM legge la struttura della `wiki/` e `raw/` — sa dove trovare concetti, fonti, skill.
4. **Skill in catalogo.** L'LLM legge `skills/` e scopre quali procedure ha a disposizione: `ingest-paper.md`, `warm-up-context.md`, ecc.
5. **In attesa.** L'LLM rimane pronto, context caricato, skill catalogate. Ti chiede: "Ho il contesto. Cosa vuoi fare?"
6. **Trigger.** Tu dici: "Ingestisci il paper su cognitive twins in `raw/papers/a.3_CogTwin/`."
7. **Esecuzione.** L'LLM richiama la skill `ingest-paper.md`, la legge, la esegue step by step — legge il paper, aggiorna la wiki, registra il log.

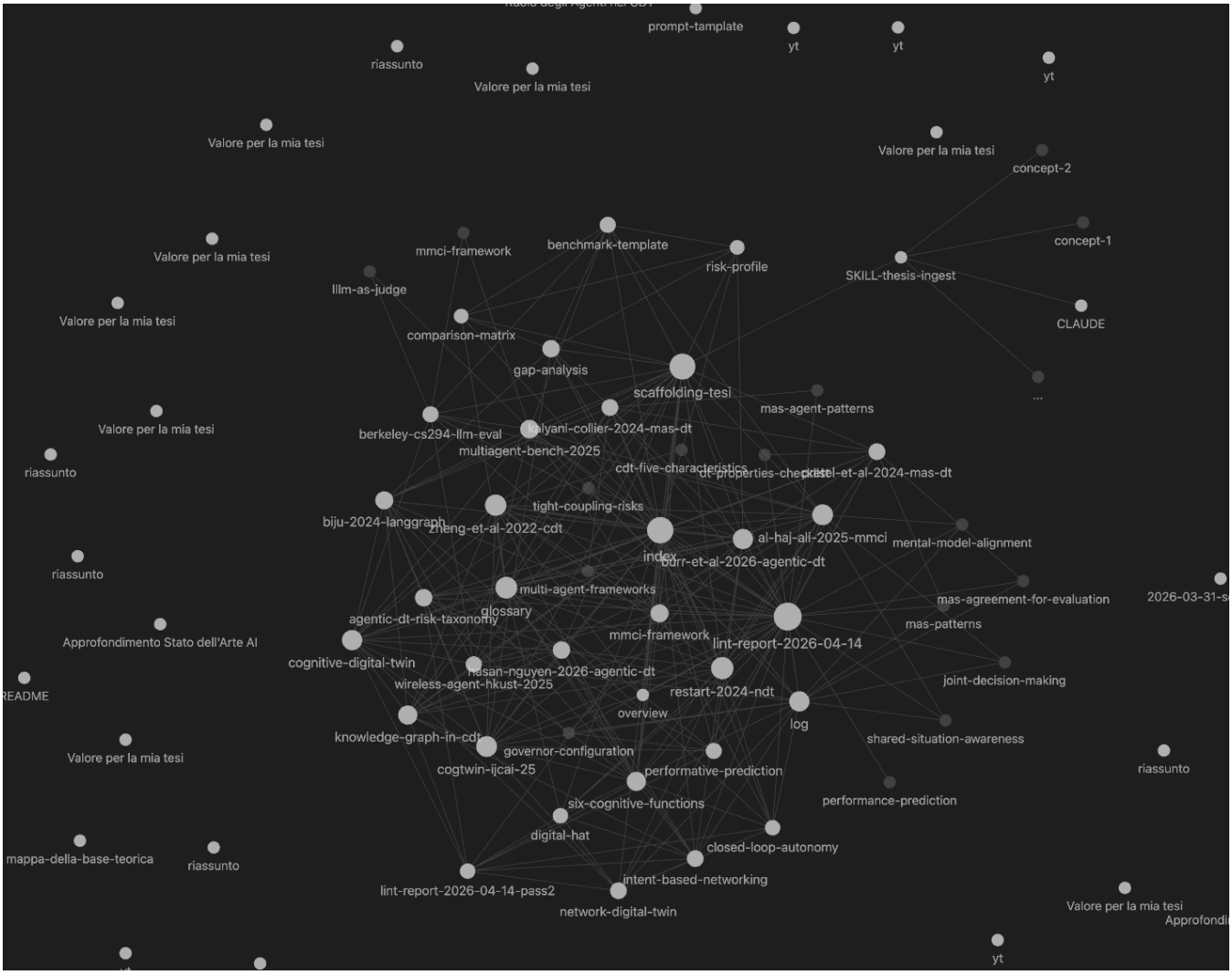
La chiave è che **nessuna comprensione del contesto va persa tra sessioni**. Ogni sessione riprende il lavoro esattamente dove lo hai lasciato. `CLAUDE.md` + skill catalog = bootstrapping istantaneo. L'LLM non deve riderivare "dove siamo" o "cosa sappiamo" — è tutto già scritto, strutturato, pronto.

Questo è il punto di arrivo del ragionamento sulla struttura invariante: la logica operativa dell'agente vive nei file, non nel framework.

6.4 Due strumenti, due ruoli

Obsidian è un gestore di file Markdown open source che costruisce automaticamente un grafo delle relazioni tra file basato sui link interni (`[[nome_file]]`). Non è un database né un CMS — è un'interfaccia visuale sopra una cartella di file. Il grafo rende navigabile la wiki lato umano,

mostrando come i concetti si collegano tra loro.



VSCode + Copilot (o qualsiasi agente con accesso al file system — Claude Code, Cursor, un agente custom) è dove avviene il lavoro computazionale. L'agente opera direttamente sul file system attraverso il terminale integrato.

Obsidian è la vista, VSCode è il motore. Il workflow funzionerebbe interamente da VSCode senza Obsidian. La combinazione dei due riflette una scelta consapevole: quando conviene interagire direttamente (navigazione, visualizzazione relazioni, lettura umana) e quando conviene delegare all'automazione (ingestione, aggiornamenti, scrittura). I file Markdown sono il layer condiviso — persistono su disco, funzionano con entrambi, e funzionerebbero con qualsiasi altro strumento che legga file di testo.

6.5 Perché funziona senza RAG

RAG è la scelta corretta in tre scenari precisi: corpus di migliaia o milioni di documenti; documenti che cambiano frequentemente; impossibilità di sintetizzare manualmente il materiale.

L'approccio wiki è la scelta corretta quando il corpus è gestibile (decine, centinaia di documenti), la qualità del ragionamento conta più della velocità di retrieval, la conoscenza deve accumularsi e raffinarsi nel tempo, e non si vuole gestire infrastruttura aggiuntiva.

Per ricerca, sviluppo su progetto specifico, studio su un dominio circoscritto — l'approccio wiki non è un ripiego. È la scelta architetturalmente corretta: produce ragionamento di qualità superiore perché l'LLM riceve conoscenza già sintetizzata e strutturata, non frammenti grezzi.

Il limite: quando il corpus supera le centinaia di documenti, la wiki stessa diventa troppo grande per essere iniettata interamente nel contesto. A quel punto RAG torna necessario — idealmente applicato alla wiki sintetizzata, non ai documenti grezzi originali.

Conclusione

La tesi di questo documento è semplice: **la struttura invariante di un agente è più importante di qualsiasi framework specifico**.

I Transformer hanno compresso la semantica umana in spazi vettoriali. Il risultato è potente ma passivo: produce testo, non agisce. Gli agenti elevano l'LLM da oracolo ad attore attraverso un loop di ragionamento e azione — ma ereditano i limiti strutturali del modello, in particolare l'assenza di memoria persistente. La memoria è il problema aperto del campo: l'approccio wiki — memoria che compila invece di recuperare — è la soluzione più efficace per domini di scala gestibile, perché produce conoscenza strutturata invece di frammenti.

I framework impacchettano questa struttura per la produzione. Sono utili, ma non sono la struttura — sono packaging sopra di essa. Capire la struttura permette di scegliere il framework, non il contrario. E quando il framework di riferimento cambia — come succede continuamente — la struttura rimane.

La dimostrazione pratica è nel workflow del capitolo 6: Obsidian per la visualizzazione, VSCode per l'esecuzione, Markdown come formato di memoria universale. Il fatto che tutto funzioni uguale con Claude Code, Copilot o qualsiasi altro agente è la conferma empirica che la struttura invariante è reale, non una semplificazione didattica.